Concurrent Programming: Languages and techniques

Making sure Rust is safe

António Ravara

(with Bernardo Toninho and Carla Ferreira)

Departament of Computer Science,
NOVA School of Science and Technology

AXM disciple too strict to do key stuff

How to deal with aliased mutable state?

Sychronisation: locks, channels, etc

Memory management: reference counting, etc

Unsafe implementations providing Safe APIs

Really? How do we know that?

Program Semantics

What is it: mathematically rigorous description of programs
One instruction at a time

Goal show programs do what they are supposed to do
Program aim needs to be formally defined

How do we show programs do the right thing?

Let's follow an old recipe to do foundational science

The book of nature is written
in the language of mathematics

Galileo Galilei, in The Assayer, 1623.

https://plato.stanford.edu/entries/galileo/

The Unreasonable Effectiveness of Mathematics
in the Natural Sciences

Eugene Wigner (Nobel Prize of Physics, 1963), in Communications in Pure and Applied Mathematics, Vol. 13, No. I (February 1960). John Wiley & Sons, Inc.

https://web.archive.org/web/20210212111540/http://www.dartmouth.edu/~matc/MathDrama/reading/Wigner.html

Foundational science in Informatics

On the Unusual Effectiveness of Logic in Computer Science

Joseph Y. Halpern, Robert Harper, Neil Immerman, Phokion G. Kolaitis, Moshe Y. Vardi and Victor Vianu, in The Bulletin of Symbolic Logic, Vol. 7, No. 2, June 2001

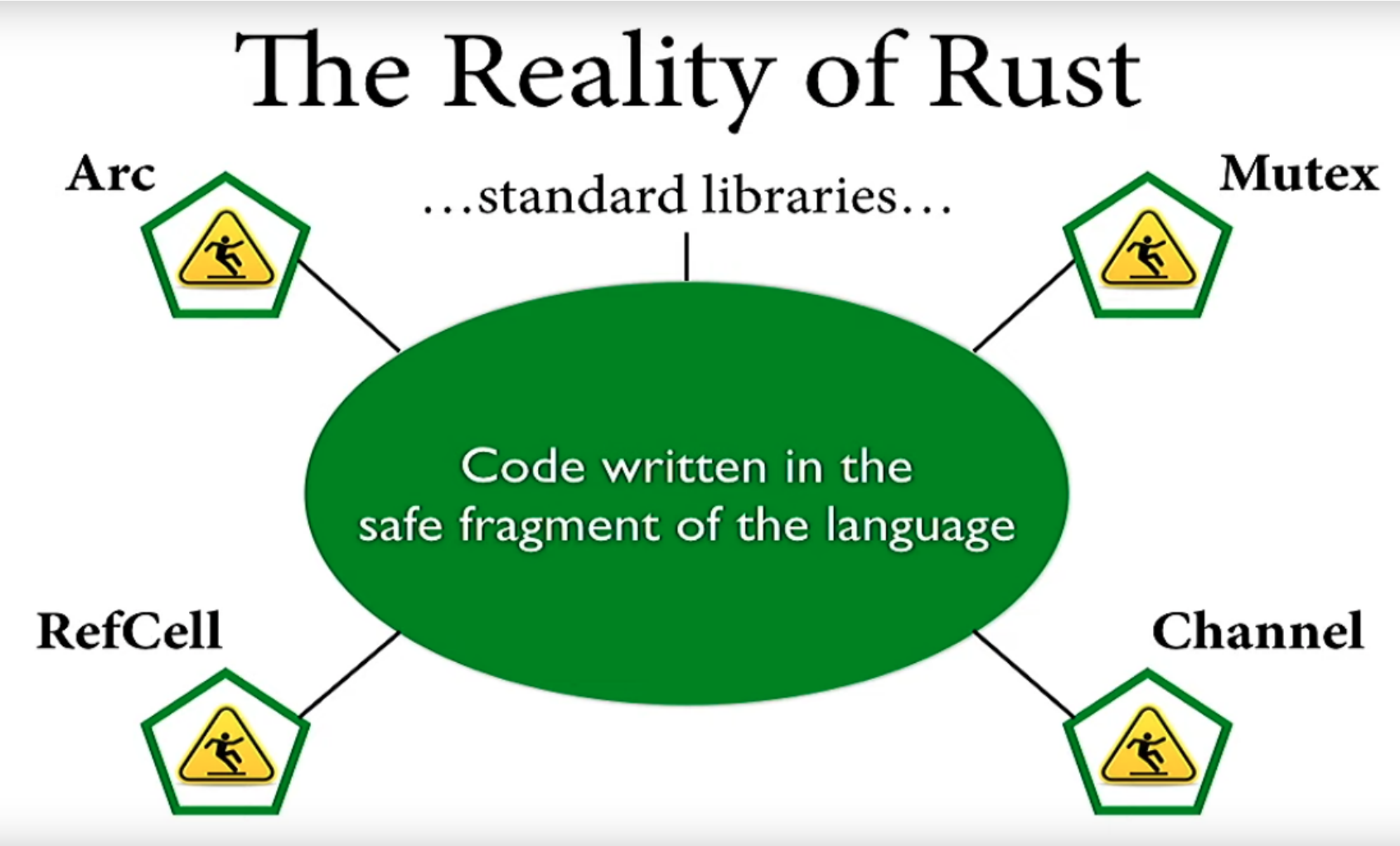
`https://www.jstor.org/stable/2687775`

Logic is the calculus of Informatics

Language to formalise computation models and machines

Reasoning systems to deduce behaviour

Rust Code is Safe?



Derek Deyer

Rust Code is not always Safe...

Leakpocalypse

Rust Issue #24292: `std::thread::JoinGuard` (and scoped) are unsound because of reference cycles

Alexis Beingessner, April 27, 2015

pause

Can the problem(s) be fixed and trust recovered?

The RustBelt project

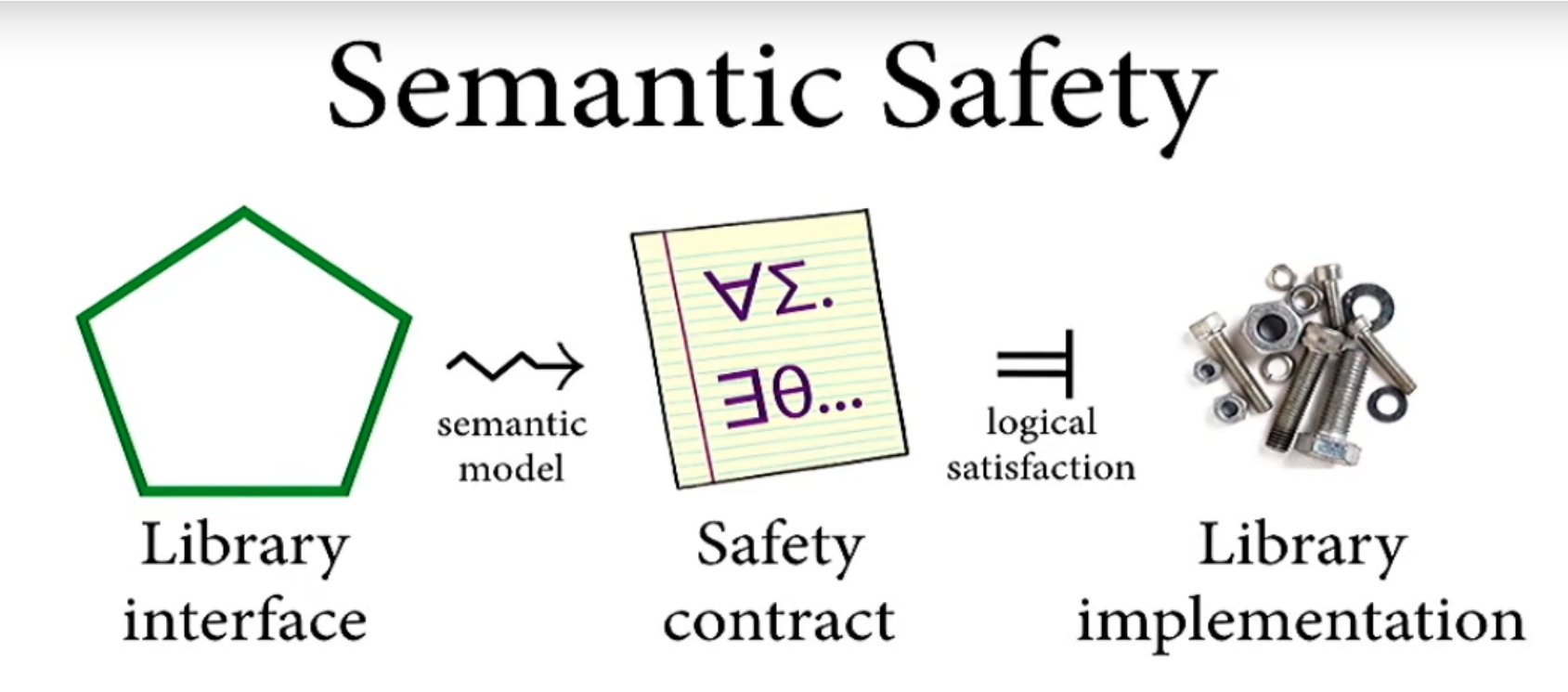
Goal develop a logical foundation for Rust

Aims
- Verify Rust type system safety
- Check the std libraries
- Provide tools to safely evolve the language

RustBelt project: showing the safety of unsafe APIs

What safety means

Well-typed code, even using "unsafe" libraries, have no undefined behaviour (memory management violations or data-races)



RustBelt project: showing the safety of unsafe APIs

What safety means

Well-typed code, even using "unsafe" libraries, have no undefined behaviour (memory management violations or data-races)

RustBelt project: showing the safety of unsafe APIs

Presentation of the Project

Web site: `https://plv.mpi-sws.org/rustbelt/`

Videos: `https://www.youtube.com/watch?v=Y9vemQmVeLI`
`https://www.youtube.com/watch?v=QLIDj-EMExU`

Back to the initial problem

How to show Rust programs do what they are supposed to do?

Even if unsafe code is used

Formal Verification Tools

Provably show a program satisfies some properties

There are many tools for (safe) Rust (mostly)

RefinedRust – a paradigmatic example

A tool to show functional correctness of safe and unsafe code

https://plv.mpi-sws.org/refinedrust/

RefinedRust: A Type System for High-Assurance Verification of Rust Programs 192:5

```
#[rr::params("x" : "Z")]
#[rr::args("#x")]
#[rr::requires("x + 42 ∈ i32")]
#[rr::returns("#(x + 42)")]
fn box_add_42(mut r: Box<i32>) -> Box<i32>
{ *r += 42; r }
```

Fig. 1. Function that adds 42 to an integer stored in a box.

```
#[rr::params("x" : "Z", "γ" : "gname")]
#[rr::args("(#x, γ)")]
#[rr::requires("x + 42 ∈ i32")]
#[rr::ensures(#iris "Res γ (x + 42)")]
fn mut_ref_add_42(r: &mut i32)
{ *r += 42 }
```

Fig. 2. Function that adds 42 to an integer stored in a mutable reference.

```
#[rr::params("xs" : "list (bor {math_type T})", "γ" : "gname", "i" : "Z")]
#[rr::args("(#xs, γ)", "i"]                                  #[rr::exists("γi")]
#[rr::returns("if i < length xs then Some (xs !!! i, γi) else None")]
#[rr::ensures(#iris "if i < length xs then Res γ (<[i:=*γi]>xs) else Res γ xs")]
fn Vec::get_mut<'a>(&'a mut self, idx: usize) -> Option<&'a mut T> {
  if idx < self.len() { unsafe { Some (self.get_unchecked_mut(idx)) } }
  else { None }
}
```